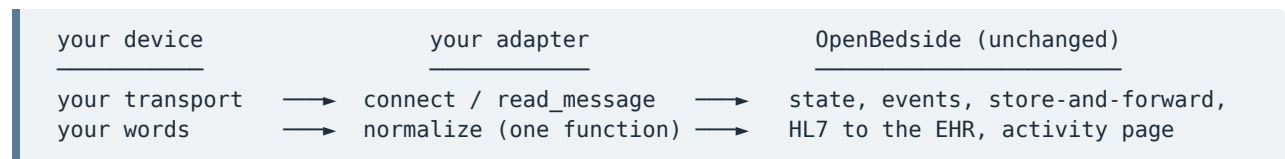


Connecting your device to OpenBedside

A guide for the engineering team at a device manufacturer. By the end you will have your own device's data flowing through OpenBedside to a stand-in hospital interface engine, checked, tested, and ready to show to a hospital's interface team.

Plan on a few days for a first working adapter if your device already exposes its data somewhere. Most of that time goes into the mapping, not the code.

The idea



Every infusion device reports the same facts: what is running, at what rate, how much is left, what drug, what state. Every device says it differently: different transport, different state names, different unit strings, different ways of showing a piggyback. Your adapter is the translation, and it is the only code you write. Everything to the right of it is the same for every manufacturer, which is the point of the project.

What your device needs to provide

At minimum, from each pump module, repeatedly while connected:

- a stable device identifier and module identifier;
- the state (running, paused, stopped, complete, KVO, alarm);
- the current rate in mL/h when running.

Better, and what a hospital will ask for: drug name, concentration, programmed rate, VTBI, volume delivered, the piggyback when one runs, dose rate with its basis, and the device's own clock. Whatever your device cannot send, write down. Missing data is a finding the hospital needs to hear from you, not discover.

Step 1. Install OpenBedside

Follow [START-HERE-install-on-your-Mac.md](#). Linux is identical. Make sure `openbedside demo` runs and the page at <http://127.0.0.1:8080> shows the simulated pump.

Step 2. Look at a finished integration first

OpenBedside ships a complete adapter for a pump that does not exist, "ACME", with an invented protocol: JSON over TCP, its own state words, its own unit strings, two channels, piggybacks. Run it:

```
openbedside fake-pump          # tab 1: the pretend device
openbedside ehr-sim           # tab 2: the pretend hospital
openbedside run -c ~/Documents/OpenBedside/config/acme-example.toml # tab 3
```

Open <http://127.0.0.1:8080>. The **Device connections** card shows the adapter connected and counting messages, but nothing is sent yet: the pump is not registered. Go to **Devices**, find ACME-7731 under *Seen but not sent*, click **Register**, and save. From then on its data goes to the stand-in hospital. Stop the fake pump (Ctrl-C in tab 1) and watch the card change and a *communication-lost* event appear; start it again and watch the adapter reconnect on its own.

Then read `src/openbedside/examples/acme_json.py`. The adapter itself is about 110 lines (the rest of the file is the fake pump) and has three parts, which is the shape your adapter will have:

1. **Vocabulary tables:** `STATE_MAP` and `UNIT_MAP`, the device's words to ours.
2. **Transport:** `connect`, `read_message`, `close`.
3. **Translation:** `normalize`, one raw message in, one snapshot out.

Step 3. Fill in the mapping worksheet

`openbedside new-adapter` (next step) puts a copy in your folder as `MAPPING.md`, and it is also at [MAPPING-WORKSHEET.md](#). Do it before code. It asks about transport, identity, every state, every value and unit, how a piggyback appears, and what your device does not send. When it is done, the adapter is mostly copying it.

The rules it enforces:

- **Never guess.** Unknown stays empty. A plausible wrong number in a medical record is worse than a blank.
- **Every number has a unit.** OpenBedside refuses a number without one.
- **A dose rate travels with its basis:** concentration, and weight if weight based. Without them, OpenBedside withholds the dose rate rather than send something the hospital cannot check.
- **A two-channel pump is one device with two modules**, never two devices.
- **The secondary is a secondary.** While a piggyback runs, the primary waits, and the channel's actual rate is the secondary's.
- **Patient identity comes from the device or not at all.** If your device holds a patient identifier (the clinician scanned the wristband at the pump), put it in `reported_patient_id`. Never look one up, never set `Device.patient`. The gateway

decides the patient from the sources the hospital allows for that device and withholds it when they disagree.

- **The same device id and vendor string every time.** The hospital registers your device by vendor plus device id before anything from it is sent.

Step 4. Generate your starter folder

```
cd ~
source ~/.venvs/openbedside/bin/activate
openbedside new-adapter widgetco --dir ~/widgetco-adapter
```

You get:

File	What it is
<code>widgetco_adapter.py</code>	your adapter, with four marked sections to fill in
<code>widgetco.toml</code>	a config that runs it, with orders switched off
<code>test_widgetco_adapter.py</code>	tests that already pass on a sample message; grow them
<code>MAPPING.md</code>	the worksheet

The folder is yours. It does not need to live inside OpenBedside, and under the Apache license you may keep it private or publish it.

Step 5. Write the four sections

Your adapter subclasses `StreamAdapter`, which handles connecting, reconnecting with backoff, counting messages, skipping bad ones without crashing, and publishing. You write:

Method	Job
<code>connect()</code>	open the connection to the device, its test harness, or your own server
<code>read_message()</code>	return the next raw message, or <code>None</code> when the connection closes
<code>normalize(raw)</code>	return a <code>Device</code> snapshot, or <code>None</code> to skip a message that is not a status report
<code>close()</code>	tidy up (optional)

Transport patterns

The device (or your server) listens on TCP and streams. This is the template as written. Change the framing in `read_message` if yours is not one JSON object per line.

The device connects to the gateway. Write `run()` yourself with a server:

```

async def run(self):
    async def on_device(reader, writer):
        while line := await reader.readline():
            dev = self.normalize(json.loads(line))
            if dev:
                self.publish(dev)
    server = await asyncio.start_server(on_device, "0.0.0.0", int(self.config["listen_port"]))
    async with server:
        await server.serve_forever()

```

Serial (RS-232 or USB serial). Install `pyserial-asyncio` into your environment and open the port in `connect()`:

```

import serial_asyncio
async def connect(self):
    self.reader, self.writer = await serial_asyncio.open_serial_connection(
        url=self.config["port"], baudrate=int(self.config.get("baud", 9600)))

```

Your company already has a device server with an API. Poll it and publish each device. Use a thread for a blocking HTTP library so the gateway stays responsive:

```

async def read_message(self):
    await asyncio.sleep(float(self.config.get("poll_seconds", 5)))
    return await asyncio.to_thread(fetch_status_json, self.config["url"]) # your function

```

Binary protocols. `read_message` returns whatever `normalize` understands: bytes, a parsed structure, a dictionary. Keep the parsing in one place and test it with recorded messages (Step 8).

Things to get right

- **Publish on every report.** Do not re-send an old snapshot to look alive. When your device goes quiet, OpenBedside needs to notice and report communication lost.
- **Keep your vocabulary in the two tables.** Anything the tables do not cover should fail loudly (an unmapped unit raises an error and the message is skipped and counted), never be passed through as a guess.
- **Report the device clock** in `device_clock` if your device has one. OpenBedside warns when it drifts from the gateway, and event order depends on it.
- **Add units to `codes.UNITS` only with a source.** Each entry carries where its MDC code came from. If your device uses a unit that is not there yet, the checker tells you, and that is a contribution worth sending back.

Step 6. Check your mapping, before any EHR

```

openbedside check -c ~/widgetco-adapter/widgetco.toml --seconds 60 --show-hl7

```

This runs your adapter against your device (or its harness) for a minute, sends nothing anywhere, and reports:

- how many snapshots arrived, from how many devices, and how many messages were skipped;
- every problem, by level: **ERROR** (will be refused or mangled), **WARNING** (will work but could mislead), **INFO**;
- the longest silence from each device, against the offline threshold;
- with `--show-hl7`, the exact PCD-01 message a hospital would receive.

It ends with **PASS** or **FAIL**. Get to PASS with no errors before Step 7.

Finding	What it means	Usual fix
wrong-unit	a rate is not mL/h, or a volume is not mL	convert in <code>normalize</code>
unit-not-mappable	no MDC code for that unit yet	map to a supported unit, or add one to <code>codes.UNITS</code> with its source
status-unknown	a device state is missing from <code>STATE_MAP</code>	add it
no-actual-rate	running or KVO with no current rate	send the rate the channel is actually delivering
dose-without-basis	dose rate with no concentration	send concentration too, or leave dose rate empty
dose-without-weight	weight-based dose with no weight	send patient weight in kg
channels-per-module	more than one delivery channel in one module	model each channel as its own module
clock-skew	device clock far from the gateway's	fix the device time source
patient-set-by-adapter	the adapter set <code>Device.patient</code>	use <code>reported_patient_id</code> , and only for an id the device itself holds
pcd01-build-failed	the HL7 message could not be built	read the message; usually a unit or structure problem

The same checks are available in your tests: `openbedside.validate.check_device(dev)`.

Step 7. Run against the stand-in hospital and watch the activity page

```
openbedside ehr-sim # tab 1
openbedside run -c ~/widgetco-adapter/widgetco.toml # tab 2
```

On <http://127.0.0.1:8080>, first register your device on the **Devices** page (it appears under *Seen but not sent*). Then:

Card	What to confirm
Status	EHR link connected; queue shows messages <i>sent</i>
Device connections	your adapter connected; messages rising; skipped at 0; no last error
Devices	one row per module, the right status, rates and volumes that match the device's own screen
Events	each thing you do at the device appears once, in order, at the right time
Outbound to EHR	every message <i>sent</i> with AA

Tab 1 prints every message as the hospital receives it. Keep it next to the device.

Step 8. Test the hard cases

Work through this list at the device, with the activity page open. Write down what you see for each. These are the situations that break integrations in hospitals.

#	Do this	Expect
1	Start an infusion	<i>delivery-start</i> ; status infusing; rate matches the device
2	Change the rate	<i>rate-change</i> with old and new values
3	Pause, then resume	<i>delivery-stop</i> , then <i>delivery-start</i>
4	Start a piggyback	a secondary source appears; actual rate becomes the secondary's
5	Let the piggyback finish	<i>secondary-to-primary</i> ; actual rate returns to the primary's
6	Let VTBI run out	<i>delivery-complete</i> and <i>kvo-start</i> ; status KVO, still infusing
7	Cause an alarm	status alarm (alarm messaging to clinicians is out of scope)
8	Power the device off	after <i>offline_after_seconds</i> , <i>communication-lost</i> ; device marked offline
9	Power it back on	adapter reconnects; <i>communication-restored</i>

#	Do this	Expect
10	Stop <code>ehr-sim</code> for two minutes, then restart it	queue grows while it is down, then drains in order (retries back off, so allow a few minutes)
11	Stop and restart the gateway mid-infusion	nothing already queued is lost
12	Set the device clock 10 minutes wrong	the checker reports <code>clock-skew</code>
13	Run two channels with different drugs	each module keeps its own drug; nothing crosses over
14	<code>openbedside ehr-sim --reject-every 5</code>	every fifth message retried, none lost, order kept
15	Run the device before registering it	listed under <i>Seen but not sent</i> ; nothing reaches the EHR
16	If your device holds a patient id: scan one	the Patients page shows it with source <i>device</i> ; PID-3 carries it
17	Register the device to a bed with ADT on, then <code>openbedside send-adt --mrn TEST0001 --unit ... --bed ...</code> with a different id than the device reports	<i>association-conflict</i> ; PID-3 is <code>Unknown</code> until they agree

Then turn what you learned into automated tests. Record real messages from your device into a file and replay them through `normalize` in `test_widgetco_adapter.py`, one test per situation above. When your device firmware changes, run the tests first.

Step 9. Show it to a hospital

When your checklist is complete, a hospital interface team will want:

- [HL7-INTERFACE.md](#): exactly what they will receive, with samples;
- your completed `MAPPING.md`, including what your device does not send;
- the output of `openbedside check --show-hl7` against your device;
- your Step 8 results.

Connect to their **test** interface engine only, with synthetic patients. OpenBedside has no encryption on its HL7 links or its web page yet, and its database is not encrypted. Until that is done, keep it on a test network and keep the page on localhost.

And again: OpenBedside is not a medical device and is not for clinical use. Anyone who takes it into a product takes on that product's regulatory obligations.

Extending to ventilators and other devices

The pieces that are the same for any bedside device stay exactly as they are: the adapter pattern, `StreamAdapter`, the outbox, delivery to the EHR, the activity page, the checker's structure.

What changes for a new device class:

1. **The data model** (`model.py`): a ventilator has settings and measured values (mode, set and measured tidal volume, rate, PEEP, FiO2, pressures) rather than infusion sources.
2. **The codes** (`codes.py`): the device class, module and metric codes for that device, from the IHE Devices Technical Framework (DEV TF-3 Rev 10.0, section 7.2, "Device: Ventilator", which tracks the IEEE 11073-10303 ventilator work) and the Rosetta Terminology Mapping, each with its source.
3. **The HL7 builder**: the containment tree for that device.
4. **A simulator**, so everyone can test without hardware.
5. **Checker rules** for that device's values and units.

The outbound message does not change: IHE's PCD-01 is not pump-specific, and DEV TF-1 names ventilators among the devices it carries. Registration already accepts a ventilator; the gateway holds its data until a ventilator data model exists, rather than send it dressed as a pump. Ventilator support is planned. If your team builds it first, it is exactly the kind of contribution this project exists for.

Keeping in step

- Update OpenBedside: pull or download the new version, then `pip install -e ~/Documents/OpenBedside` again.
- Your adapter depends only on `openbedside.model`, `openbedside.adapters.stream` and `openbedside.validate`. Those are kept stable; changes will be listed in `CHANGELOG.md`.
- Found a problem in OpenBedside itself, or added a unit, a check or a transport pattern others could use? See [CONTRIBUTING.md](#).